

Структура билета

Вариант алгоритма X

1. В заданном алгоритме выделен инвариант. Ввести конструктив для алгоритма.
2. Доказать полноту и корректность алгоритма на основе конструктивной логики.
3. Доказать полноту и корректность алгоритма на основе модальной логики.
4. Доказать полноту и корректность алгоритма на основе нечеткой логики.
5. Доказать полноту и корректность алгоритма на основе временной логики.
6. Доказать полноту и корректность алгоритма на основе вероятностной логики.

При доказательстве используйте логические операции (конъюнкция, дизъюнкция, импликация, эквивалентности, отрицание)

Примеры алгоритмов X.

Алгоритмы сортировки

1. Сортировка вставками (Insertion Sort):
 - Инвариант: На каждой итерации элементы перед текущим индексом отсортированы. Каждое новое значение вставляется в правильную позицию, что гарантирует, что массив остается отсортированным.
2. Сортировка выбором (Selection Sort):
 - Инвариант: На каждой итерации в начале массива находятся все минимальные элементы, тем самым сокращая неотсортированную часть.
3. Бурная сортировка (Bubble Sort):
 - Инвариант: На каждой итерации элемент, находящийся в конечной позиции, является наибольшим из оставшихся.
4. Сортировка слиянием (Merge Sort):
 - Инвариант: Сначала массив разбивается на две части, каждая из которых отсортирована; последующее объединение отсортированных частей гарантирует глобальную сортировку.
5. Быстрая сортировка (Quick Sort):
 - Инвариант: После каждой итерации элементы разделены по отношению к опорному элементу, и рекурсивное применение сохраняет эту отделенность.
6. Сортировка кучей (Heap Sort):
 - Инвариант: Куча строится и поддерживается в правильной форме, что обеспечивает извлечение максимального элемента.

Алгоритмы поиска

7. Линейный поиск (Linear Search):

- Инвариант: Каждый элемент проходит проверку до тех пор, пока не будет найден нужный элемент или не будет достигнут конец массива.

8. Бинарный поиск (Binary Search):

- Инвариант: На каждой итерации массив делится пополам, исключая половину, где не может находиться искомый элемент.

Алгоритмы поиска в графах

9. Алгоритм Дейкстры (Dijkstra's Algorithm):

- Инвариант: На каждом шаге к вершинам, для которых найдены кратчайшие пути, не могут быть найдены более короткие пути, чем уже известные.

10. Алгоритм Флойда-Уоршелла (Floyd-Warshall Algorithm):

- Инвариант: На каждом шаге обновляются минимальные расстояния между всеми парами вершин, основываясь на найденных путях.

11. Алгоритм Прима (Prim's Algorithm):

- Инвариант: Поддерживает минимальное остовное дерево по мере добавления новых ребер с минимальным весом.

12. Алгоритм Краскала (Kruskal's Algorithm):

- Инвариант: Поддерживает множество уже добавленных ребер, гарантируя, что новые ребра не образуют циклы.

Алгоритмы динамического программирования

13. Алгоритм Кнута-Морриса-Пратта (KMP Algorithm):

- Инвариант: Использует информацию о частичных совпадениях для ускорения поиска, предотвращая ненужные повторные проверки.

14. Наибольшая общая подпоследовательность:

- Инвариант: Вычисляет длину поэтапно, основываясь на уже решенных подзадачах, что гарантирует корректный результат.

15. Задача о рюкзаке (0/1 Knapsack Problem):

- Инвариант: Поддерживает наилучшие решения для меньших подзадач, которые могут быть использованы для решения целевой задачи.

16. Минимальное расстояние Левенштейна:

- Инвариант: Сохраняет расстояния для всех подстрок, что позволяет получить минимальное количество операций.

17. Алгоритм К-ойредакции (K-Edit Distance):

- Инвариант: Поддерживает таблицу с расстояниями между всеми подстроками, где на каждой итерации обновляются минимальные расстояния с учетом добавления, удаления или замены символов. Это гарантирует, что для каждой пары подстрок вычисляется правильное расстояние.

18. Задача о наибольшей общей подпоследовательности (Longest Common Subsequence):

- Инвариант: Сохраняет длину наибольшей общей подпоследовательности для всех пар префиксов исходных строк. Каждое новое значение в таблице

зависит от уже рассчитанных значений, что обеспечивает правильность конечного результата.

Другие алгоритмы

19. Алгоритм Хаффмана (Huffman Coding):

- Инвариант: Строит дерево, корректно присваивая коды, так что более частые символы получают более короткие коды.

20. Обход в глубину (DFS):

- Инвариант: Обходит все смежные вершины перед переходом к следующей, обеспечивая полное покрытие всех вершин.

Алгоритмы сортировки и поиска

21. Сортировка пирамидой (Heap Sort):

- Инвариант: На каждой итерации поддерживается свойство кучи, что гарантирует, что максимальный (или минимальный) элемент всегда находится на вершине. После извлечения этого элемента он добавляется в отсортированный массив, что обеспечивает, что оставшиеся элементы остаются в структуре кучи.

22. Троичный поиск (Ternary Search):

- Инвариант: Работает аналогично бинарному поиску, но делит массив на три части. На каждом шаге два порога используются для уменьшения рабочей области, что гарантирует, что искомый элемент будет найден за конечное число шагов при условии, что массив отсортирован.

Алгоритмы для дерева

23. Обход в ширину (Breadth-FirstSearch, BFS):

- Инвариант: Обходит все вершины на текущем уровне дерева или графа, прежде чем перейти к следующему уровню, обеспечивая тем самым, что все узлы посещены в порядке их близости к начальной вершине.

Алгоритмы работы с потоками

24. Алгоритм Форда-Фалкерсона (Ford-Fulkerson Algorithm):

- Инвариант: Поддерживает текущий поток в графе, используя пути увеличения, чтобы находить максимальный поток от источника к стоку. Для каждого найденного пути увеличивается поток до тех пор, пока не будут исчерпаны возможности увеличения, тем самым гарантируя, что конечный результат будет максимальным.

+++++